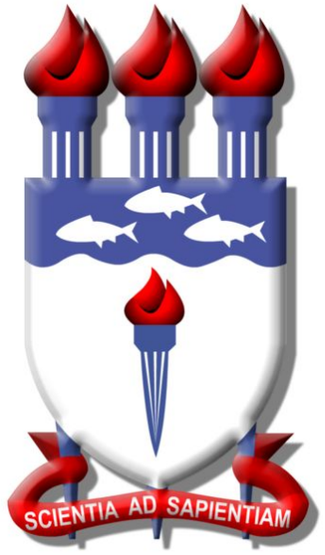




Árvore B

Laryssa Barreto Vasconcelos Calazans, Murilo Duarte dos Santos, Narel Silva de Oliveira e Rafael Antonio da Silva Santos

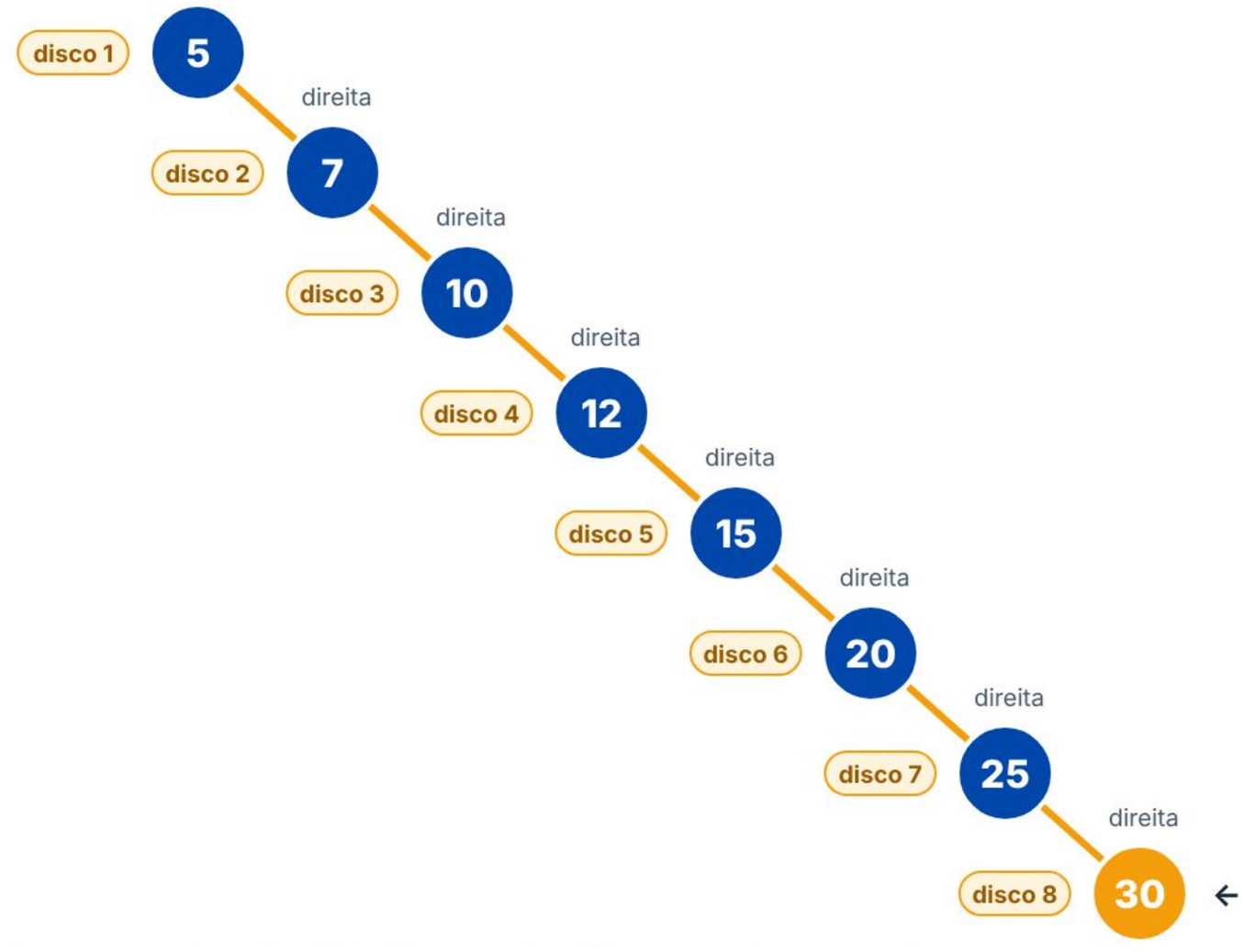
https://github.com/narelIC/Projeto-ED-2026-2

Necessidade

- Como **reduzir o tempo** de **leitura** de um Banco de Dados?

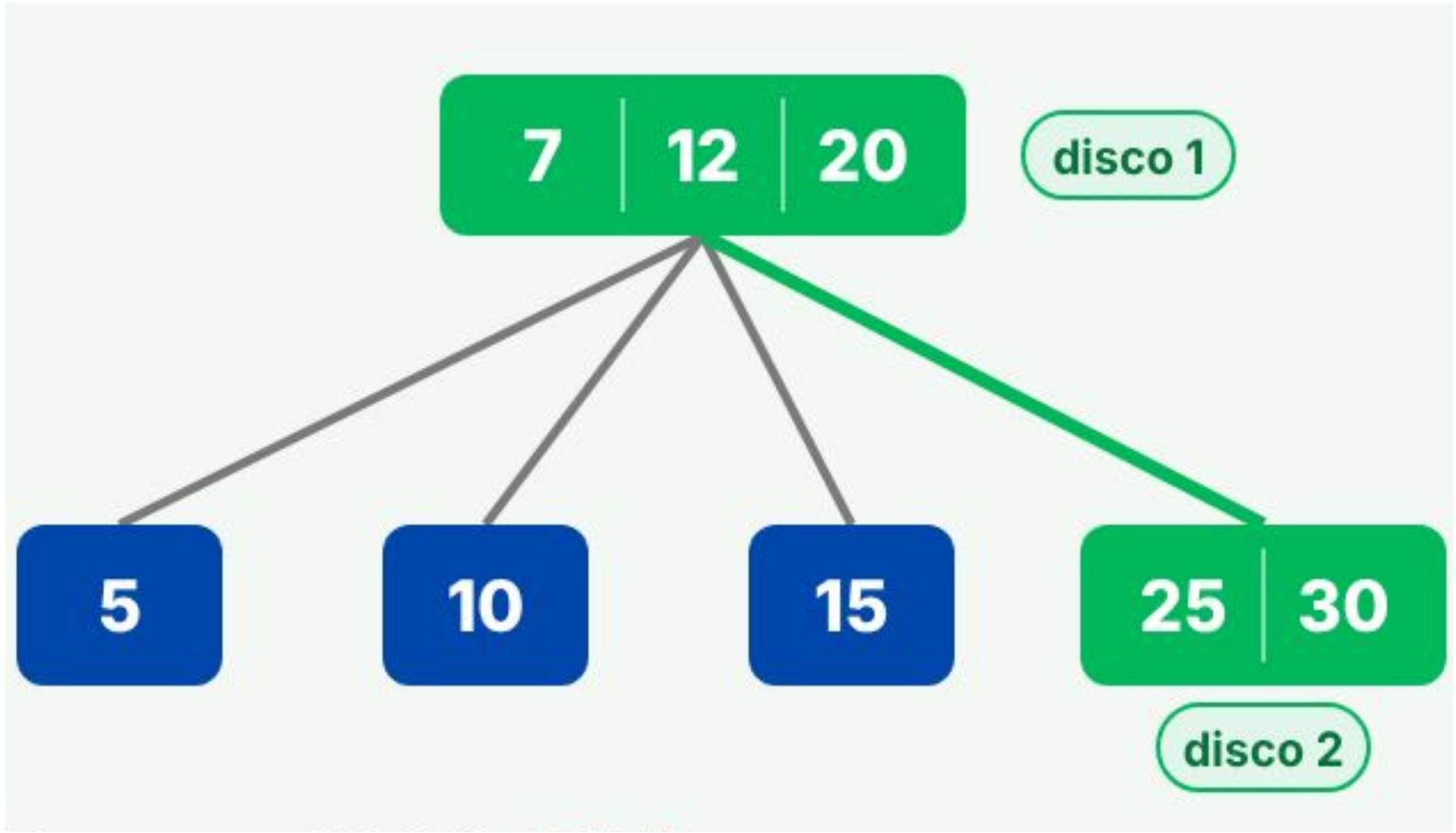
- **Sistemas de arquivos** como o NTFS usam Árvore B ou B+**;**

- Em **Banco de Dados** é estrutura padrão dos **índices**.



A Árvore B

- Uma ED árvore, auto-balanceada, de múltiplos caminhos que permite que cada pai tenha múltiplos filhos e cada filho tenha múltiplos pais. Com isso sua profundidade é menor que uma árvore de busca binária. Ex:



Como foi implementado?

- Criar Nó;

```
No *criarNo(int folha)
```

- Inserir;

```
void inserir(No **raizRef, int valor)
```

- Apagar;

```
No *buscar(No *raiz, int valor)
```

- Destruir.

```
void liberarArvore(No *no)
```

Algoritmo de Dijkstra

Laryssa Barreto Vasconcelos Calazans, Murilo Duarte dos Santos, Narel Silva de Oliveira e Rafael Antonio da Silva Santos

https://github.com/narelIC/Projeto-ED-2026-2

Necessidade

- Qual a **menor distância** entre duas cidades?

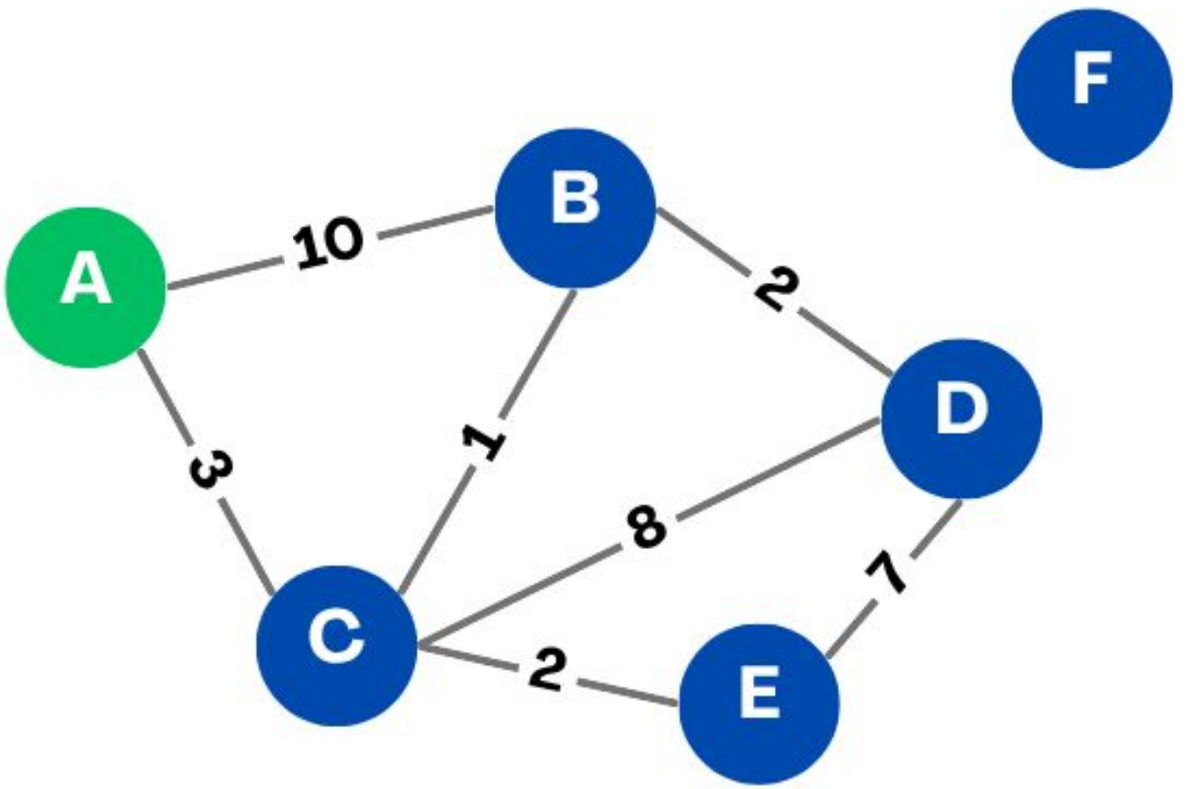
- Roteamento de **redes** de computadores como **OSPF (Open Shortest Path First)**;



- **Jogos** digitais para **pathfinding** de personagens não jogáveis (NPCs).

O Algoritmo de Dijkstra

- Encontra a menor distância de um vértice de origem até todos os outros vértices de um grafo ponderado não direcionado. Ex:



Origem	A	0
Vértice	B	1
Vértice	C	2
Vértice	D	3
Vértice	E	4
Vértice	F	5

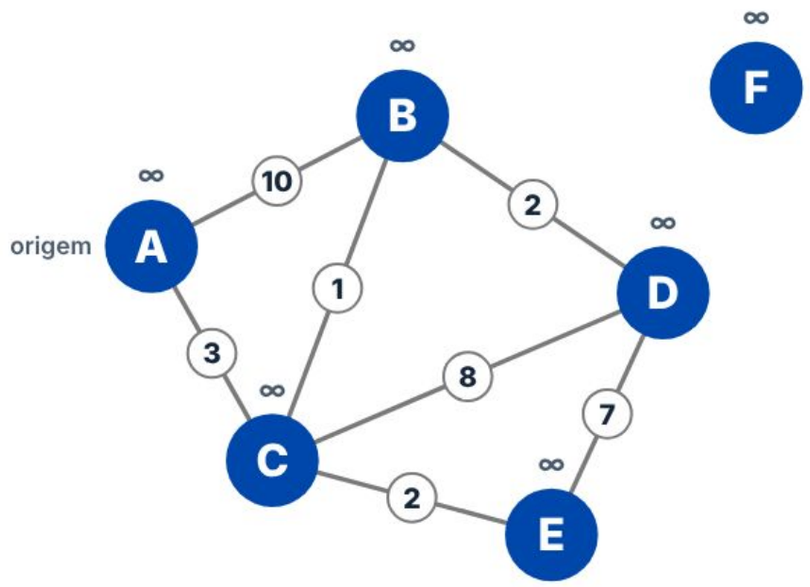
Como foi implementado?

- Matriz de Adjacência;

```
int grafo[V][V]
```

- Vetores auxiliares;

```
int dist[V];
int visitado[V];
```



Vértice	Índice	dist	visitado
A	0	∞	0
B	1	∞	0
C	2	∞	0
D	3	∞	0
E	4	∞	0
F	5	∞	0

não visitado u (escolhido) visitado

- Algoritmo de busca.

```
for (int i = 0; i < V; i++) {
    if (!visitado[i] && dist[i] < menor) {
        menor = dist[i];
        u = i;
    }
}
```

Código em C

```c
void dijkstra(int grafo[V][V], int origem) {
    int dist[V];
    int visitado[V];

    // Inicializa as distâncias e os vértices visitados
    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        visitado[i] = 0;
    }

    // A distância da origem para ela mesma é zero
    dist[origem] = 0;

    // Processa os vértices
    for (int cont = 0; cont < V - 1; cont++) {
        int menor = INF;
        int u = -1;

        // Encontra o vértice não visitado
        // que possui a menor distância conhecida
        for (int i = 0; i < V; i++) {
            if (!visitado[i] && dist[i] < menor) {
                menor = dist[i];
                u = i;
            }
        }

        // Se não há mais vértices alcançáveis, encerra
        if (u == -1)
            break;

        // Se não há mais vértices alcançáveis, encerra
        if (u == -1)
            break;

        visitado[u] = 1;

        // Atualiza as distâncias dos vizinhos de u
        for (int v = 0; v < V; v++) {
            if (!visitado[v] &&
                grafo[u][v] != 0 &&
                dist[u] + grafo[u][v] < dist[v]) {

                dist[v] = dist[u] + grafo[u][v];
            }
        }
    }

    // Exibe as menores distâncias encontradas
    printf("Vertice | Distancia da origem\n");

    for (int i = 0; i < V; i++) {
        if (dist[i] == INF)
            printf("%d       | Inalcancavel\n", i);
        else
            printf("%d       | %d\n", i, dist[i]);
    }
}
```

De volta à Motivação…

- Por que o algoritmo resolveu o problema descrito na motivação?!

